Adaptive Pass Bounds without a Runtime Assumption

Hans Brende GPT-6 Astra (OpenAI)

October 2026

Abstract

The adaptive pass theorem in *Matrix Transposition and the Tape Coding Conjecture* yields a logarithmic pass lower bound under a polynomial runtime hypothesis. We remove that hypothesis for halting machines in its exact formal model. A native run of length T on an L-bit input, partitioned into r monotone passes, satisfies $L + T + 1 \le (L + 1)C_M^r$, with an explicit machine constant C_M. Substitution into the original information bound gives $r \ge \log_{k+1} d - O_M(1)$ at logarithmic entry width, for $k \ge 1$ work tapes. A grouped bound also applies to a specified set of correct halting inputs. All new numbered results are verified in Lean. The run-length principle is classical; the contribution is an explicit proof in the original model and its checked application to the adaptive pass theorem.

Contributions. Hans Brende directed the research. GPT-6 Astra (OpenAI) developed the argument, formalization, and text under his direction. This memo extends the first paper by Hans Brende and Claude Opus 5.5 (Anthropic), reusing its machine definitions and adaptive information theorem.

Contents

1 Purpose and model **1**

2 Native run length **2**

3 Removing time from the adaptive pass bound **3**

4 Consequences and remaining problem **4**

A Index of Lean declarations **5**

B Building and verification **5**

1 Purpose and model

The first paper's Theorem P′ bounds the information delivered by an adaptive machine using few monotone passes [1]. Its explicit inequality includes the runtime T. The paper obtains $r \ge \log_{k+1} d - O_M(1)$ when $\log T = O(b)$ and identifies removal of the runtime premise as unfinished formal work. This memo supplies that step for halting runs. It leaves the original finite-time and nonhalting-prefix statements intact.

Bounds exponential in the number of reversals are standard. In particular, Hernich and Schweikardt state a run-length bound for reversal- and space-bounded machines in Lemma 2.3,

attributing the deterministic argument to earlier work [2]. Our proof works directly with the first paper's deterministic machine, including simultaneous head moves, stationary heads, and unreadable output. We make no priority claim for the run-length principle.

The model has $q+1$ states, a read-only binary input tape, k initially blank work tapes with $g+1$ symbols, and an append-only output that the machine cannot read. Each input or work head moves by -1, 0, or 1 per step; all start at position zero. A pass is an interval on which each of these $k+1$ head trajectories is monotone, with a separate direction for each head. Empty passes are allowed. A run halting at T has a transition at every step before T and none at T.

We use d for the matrix dimension, b for its entry width, and $L = d^2 b$. The canonical width is $b(d) = \max(1, \lceil \log_2 d \rceil)$, as in the frozen statement of the first paper. For one fixed machine define

$$S_M = 3(q+1)(g+1)^k, \qquad C_M = (2k+3)(S_M+1), \tag{1.1}$$

$$\sigma_M = \lceil \log_2(q+1) \rceil + k \lceil \log_2(g+1) \rceil, \qquad \ell_M = \lceil \log_2 C_M \rceil,$$

$$A_M = 2 + \sigma_M + (k+4)(5+\ell_M). \tag{1.2}$$

The Lean declarations are authoritative proofs; the outlines below explain them. This memo imports the original library by a relative path and does not alter its frozen machine definitions.

2 Native run length

Lemma 2.1 (Clean-ray intervals)**.** Lean: `ray_simulation`, `ray_eventually_none`, `ray_interval_bound` [`RaySimulation.lean`]

At the start of an interval, assign each head either the instruction to stay at its current cell or a direction along a ray. For a moving input head, assume the whole ray lies outside the input word. For a moving work head, assume every cell strictly ahead on the ray is blank. During the interval, allow each head only stationary moves or moves along its assigned ray. If the actual run eventually halts, any such interval of executed steps has length at most S_M.

Outline of the formal proof. The signature consists of the state, the currently scanned input symbol (including the outside-word symbol), and the k scanned work symbols. There are exactly S_M signatures. A stationary work head next sees its written symbol; an advancing work head sees blank. A permitted input move preserves the scanned symbol. These facts define an exact finite transition system, with an exit when the native transition is undefined or attempts a forbidden direction. The invariant about blank rays is preserved by writes.

The formal simulation lifts every nonexit finite transition to the actual machine step. Consequently eventual native halting forces eventual exit from the finite system. Before the first exit its signatures are distinct: a repeated signature would repeat the future trajectory and prevent exit. The interval length is therefore bounded by the number of signatures. The output is reproduced by the native step but need not be in the signature, since no transition reads it. □

Lemma 2.2 (One-pass envelope)**.** Lean: `pass_envelope` [`RunLength.lean`]

Suppose the run halts at T, $0 \le a \le e \le T$, and $[a, e)$ is a pass. For $L = |w|$,

$$L + e + 1 \le C_M(L + a + 1).$$

Outline of the formal proof. Put $R = L + a$. Every cell visited before a, and every nonblank input cell, lies in $[-R, R]$. Mark a step if some head moves and its departing or arriving cell lies in this interval. A monotone head has at most $2R+2$ marked steps, so the total number m is at most $(k+1)(2R+2)$.

Between marked steps, a head inside the envelope remains stationary. A head outside it can move only outward, since it started the pass inside and is monotone. Its forward work cells are blank: they were visited neither before a nor earlier in the pass. The input head outside the envelope sees only the outside-word symbol. Thus each unmarked interval satisfies Lemma 2.1. The m marked steps split $[a, e)$ into at most $m + 1$ unmarked intervals, giving $e - a \le (m + 1)S_M + m$. The displayed bound follows from Eq. (1.1). □

Theorem 2.3 (Halting run-length bound)**.** Lean: `halting_run_length`, `halting_time_le`, `timeBound_log` [`RunLength.lean`]

If the actual run on an L-bit word halts at T and has a partition into r passes, then

$$L + T + 1 \le (L + 1)C_M^r. \tag{2.1}$$

In particular, with $\overline{T} = (L + 1)C_M^r$,

$$T \le \overline{T}, \qquad \lceil \log_2(2\overline{T} + 1) \rceil \le 2 + \lceil \log_2(L + 1) \rceil + r\ell_M. \tag{2.2}$$

Outline of the formal proof. Apply Lemma 2.2 to successive partition intervals, starting with $L+0+1$. This gives $L + p_i + 1 \le (L + 1)C_M^i$ at every endpoint p_i. At $p_r = T$ it proves Eq. (2.1). For Eq. (2.2), bound $L + 1$ and C_M by their respective powers of two and use $2\overline{T} + 1 \le 4\overline{T}$, since $\overline{T} \ge 1$. □

This is a restriction on how much computation a halting construction can hide in a small number of passes. It is not a lower bound on time itself.

3 Removing time from the adaptive pass bound

Write $H_r = \sum_{i=0}^{r-1}(k + 1)^i$, so $kH_r + 1 = (k + 1)^r$, and let $G_\gamma = \lceil d/\gamma \rceil$ for $\gamma \ge 1$. Set

$$B_M(L, r) = \sigma_M + (k + 4)(2 + \lceil \log_2(L + 1) \rceil + r\ell_M).$$

Theorem 3.1 (Time-free grouped bound)**.** Lean: `time_free_grouped`, `time_free_grouped_on`, `time_free_div` [`AdaptivePasses.lean`]

Suppose M correctly transposes every $d \times d$ input with b-bit entries, and every input's halting run has a partition into r passes. Then

$$d^2b \le H_r((\gamma + 1)db + G_\gamma B_M(d^2b, r)). \tag{3.1}$$

In particular $db \le H_r(2b + B_M(d^2b, r))$.

More generally, let W be a set of inputs on each of which the machine halts with the correct transposed output and an r-pass partition. If $2^{d^2b} \le 2^m|W|$, then

$$d^2b \le G_\gamma m + H_r((\gamma + 1)db + G_\gamma B_M(d^2b, r)). \tag{3.2}$$

No condition is imposed on inputs outside W.

Outline of the formal proof. Theorem 2.3 supplies the common bound $\overline{T} = (d^2b + 1)C_M^r$ on all relevant halting times. Apply the existing Lean-verified grouped Theorem P′ (or its correct-set form) from the first paper, and bound its term $\lceil \log_2(2\overline{T} + 1) \rceil$ using Eq. (2.2). The correct-set form keeps the original payment $G_\gamma m$. The per-d form uses the original theorem's corresponding arithmetic consequence. □

Theorem 3.2 (Logarithmic passes without a runtime assumption)**.** Lean: `logarithmic_pass_power`, `canonical_pass_power`, `canonical_logarithmic_passes` [`AdaptivePasses.lean`]

Let $k \geq 1$, $b \geq 1$, and $d \leq 2^b$. Under the full-correctness and halting hypotheses of Theorem 3.1,

$$kd \leq A_M(k+1)^r. \tag{3.3}$$

For the canonical width $b = b(d)$ and $d \geq 1$, this gives

$$r \geq \log_{k+1} d - (\log_{k+1} A_M - \log_{k+1} k).$$

The subtracted quantity depends only on the fixed machine.

Outline of the formal proof. Because $b < 2^b$ and $d \leq 2^b$, one has $d^2b + 1 \leq 2^{3b}$. If $r \leq b$, then $2b + B_M(d^2b, r) \leq A_M b$. The per-$d$ bound gives $d \leq A_M H_r$; multiply by k and use $kH_r + 1 = (k+1)^r$. If $r > b$, then $d \leq 2^b \leq (k+1)^r$, and $A_M \geq k$ proves the same conclusion. The canonical width satisfies $b(d) \geq 1$ and $d \leq 2^{b(d)}$. Taking logarithms in Eq. (3.3) proves the final inequality. □

Corollary 3.3 (The earlier runtime-restricted consequence)**.** Lean: `bounded_runtime_corollary` [`AdaptivePasses.lean`]

The conclusion Eq. (3.3) at canonical width still holds if the hypotheses additionally give a common runtime bound T_0. In particular, the earlier logarithmic-pass consequence for polynomial-time machines follows without using its polynomial bound.

Outline of the formal proof. Discard the bound $T \leq T_0$ and apply Theorem 3.2. This rederives the runtime-restricted consequence, not the original finite-time inequality, which is an input to our proof. □

4 Consequences and remaining problem

An adaptive transposer cannot evade the logarithmic-pass barrier by using very long monotone stretches between direction changes. The grouped correct-set inequality also removes the need to assume a common runtime on the specified halting inputs. Its co-entropy cost is retained explicitly; we do not assert an additional robust asymptotic bound here.

These results do not decide (T), the existence of a fixed finite-tape machine transposing the canonical inputs in $O(d^2b(d))$ time. A linear-time run can still have many short passes. A stronger impossibility argument must turn the required passes into enough total work, while a construction must exploit them within its time budget. The finite-time P′ theorem also applies to certain nonhalting prefixes; our new run-length argument requires eventual halting and does not replace that broader interface.

A Index of Lean declarations

Declarations are relative to `PassMemo`; their axiom checks are generated together with this index (Appendix B).

`PassMemo.AdaptivePasses` `bounded_runtime_corollary`, `canonical_logarithmic_passes`, `canonical_pass_power`, `logarithmic_pass_power`, `time_free_div`, `time_free_grouped`, `time_free_grouped_on`.

`PassMemo.RaySimulation` `ray_eventually_none`, `ray_interval_bound`, `ray_simulation`.

`PassMemo.RunLength` `halting_run_length`, `halting_time_le`, `pass_envelope`, `timeBound_log`.

B Building and verification

The library uses Lean and Mathlib v4.34.1 and imports the first paper's `transpose` package through `../../lean`. Clone the complete repository, retaining the sibling directories. From this memo's `lean/` directory run

```
lake exe cache get
lake build
lake env lean PaperAxioms.lean
```

From the memo directory, `python3 tools/lean_index.py` regenerates both the preceding declaration index and the axiom-check file from the LaTeX references. The cited proofs use at most `propext`, `Classical.choice`, and `Quot.sound`; no additional axioms or kernel-bypassing evaluation are used. The original frozen files are unchanged. Build the PDF in `paper/` with `latexmk -pdf main.tex`.

References

[1] Hans Brende and Claude Opus 5.5 (Anthropic). Matrix transposition and the tape coding conjecture. Manuscript and Lean formalization, repository HansBrende/papers, 2026. Pinned revision b6aaab17c9458e9ca39b35bd29a060910b71fdbe. URL: https://github.com/HansBrende/papers/tree/b6aaab17c9458e9ca39b35bd29a060910b71fdbe/Matrix-Transposition-and-the-Tape-Coding-Conjecture.

[2] André Hernich and Nicole Schweikardt. Reversal complexity revisited. arXiv:cs/0608036v1, 2006. Lemma 2.3; submitted 7 August 2006. URL: https://arxiv.org/abs/cs/0608036v1.